

**UNLP – Facultad de Informática**  
**Algoritmos y Estructuras de Datos**  
**Práctica 4 – Análisis de Algoritmos – Resolución completa**

## Ejercicio 1 – Procesamiento de movimientos bancarios

### a) Tabla de tiempos empíricos (ejecutados)

| N (cuentas y consultas) | procesarMovimientos | procesarMovimientosOptimizado |
|-------------------------|---------------------|-------------------------------|
| 1.000                   | 0.032 seg           | 0.001 seg                     |
| 10.000                  | 0.517 seg           | 0.005 seg                     |
| 25.000                  | 3.49 seg            | 0.017 seg                     |
| 50.000                  | 15.4 seg            | 0.016 seg                     |
| 100.000                 | 71.0 seg            | 0.015 seg                     |

Conclusión: cuando N se duplica, procesarMovimientos tarda aproximadamente 4 veces más (comportamiento cuadrático  $O(N^2)$ ), mientras que procesarMovimientosOptimizado se mantiene prácticamente constante a esta escala ( $O(N)$ ).

### b) ¿Por qué procesarMovimientos es ineficiente?

El algoritmo posee dos bucles anidados: el externo recorre todas las consultas (Q veces) y el interno recorre el rango de cuentas de cada consulta (hasta N veces). Esto implica que en el peor caso se realizan  $Q \times N$  operaciones. Como  $Q = N$  en este ejercicio, la complejidad es  $O(N^2)$ .

Con millones de consultas que abarcan gran parte de las cuentas, el tiempo crece de forma cuadrática, volviéndolo inviable para grandes volúmenes de datos.

### c) ¿En qué se diferencia procesarMovimientosOptimizado?

En lugar de recorrer todo el rango de cuentas por cada consulta, el algoritmo optimizado utiliza un arreglo auxiliar y realiza solo 2 operaciones por consulta:

```
aux[consulta.desde] += consulta.valor
aux[consulta.hasta+1] -= consulta.valor
```

Luego hace un único recorrido del arreglo auxiliar acumulando los valores y actualizando las cuentas. De esta forma, cada consulta cuesta  $O(1)$  y el recorrido final es  $O(N)$ , resultando en una complejidad total de  $O(Q + N) = O(N)$ .

## Ejercicio 2 – Búsqueda lineal vs dicotómica

### Tabla de resultados empíricos (ejecutados, peor caso)

| N       | Lineal (iteraciones) | Lineal (tiempo) | Dicotómica (iteraciones) | Dicotómica (tiempo) |
|---------|----------------------|-----------------|--------------------------|---------------------|
| 100.000 | 100.000              | 1 ms            | 16                       | 0 ms                |
| 200.000 | 200.000              | 0 ms            | 17                       | 0 ms                |
| 300.000 | 300.000              | 5 ms            | 18                       | 0 ms                |
| 400.000 | 400.000              | 4 ms            | 18                       | 0 ms                |
| 500.000 | 500.000              | 0 ms            | 18                       | 0 ms                |
| 600.000 | 600.000              | 0 ms            | 19                       | 0 ms                |

La búsqueda lineal recorre todos los elementos en el peor caso:  $O(N)$ . La dicotómica divide el problema a la mitad en cada paso:  $O(\log_2 N)$ . Para  $N = 600.000$ , la lineal hace 600.000 iteraciones mientras que la dicotómica hace solo 19 ( $\log_2(600.000) \approx 19$ ).

### Ejercicio 3 – ArrayList: $O(1)$ vs $O(N)$

El ArrayList internamente es un array con un campo que guarda el tamaño actual.

**Operaciones  $O(1)$  – tiempo constante:** `size()`, `isEmpty()`, `get(i)`, `set(i, valor)`, `iterator()`, `listIterator()`.

Estas operaciones no dependen del tamaño de la lista: `size()` devuelve directamente una variable entera ya guardada; `get(i)` y `set(i)` acceden a la posición  $i$  del array interno de forma directa (acceso por índice en memoria contigua), sin necesidad de recorrer nada.

**Operaciones  $O(N)$  – tiempo lineal:** `add(i, valor)`, `remove(i)`, `contains(valor)`.

Estas requieren tocar múltiples elementos: `add` o `remove` en el medio obligan a desplazar todos los elementos posteriores; `contains` debe recorrer el array comparando uno a uno hasta encontrar el elemento o llegar al final.

### Ejercicio 4 – Verdadero o Falso con Big-Oh

**a)  $3^n$  es  $O(2^n)$  → FALSO**

Para que  $f(n)$  sea  $O(g(n))$  debe existir una constante  $C$  tal que  $f(n) \leq C \cdot g(n)$  para  $n$  suficientemente grande. Pero  $3^n / 2^n = (3/2)^n \rightarrow \infty$ , es decir,  $3^n$  crece más rápido que cualquier múltiplo constante de  $2^n$ . No existe tal  $C$ .

**b)  $n + \log_2(n)$  es  $O(n)$  → VERDADERO**

$\log_2(n) \leq n$  para todo  $n \geq 1$ , por lo tanto  $n + \log_2(n) \leq n + n = 2n$ . Tomando  $C = 2$  y  $n_0 = 1$ , se cumple  $n + \log_2(n) \leq 2 \cdot n$  para todo  $n \geq 1$ . Luego es  $O(n)$ .

**c)  $n^{1/2} + 10^{20}$  es  $O(n^{1/2})$  → VERDADERO**

$10^{20}$  es una constante. Para  $n$  suficientemente grande ( $n \geq 10^{40}$ ),  $n^{1/2} \geq 10^{20}$ , por lo que  $n^{1/2} + 10^{20} \leq 2 \cdot n^{1/2}$ . Tomando  $C = 2$  y  $n_0 = 10^{40}$ , se cumple la definición. Es  $O(n^{1/2})$ .

**d)  $f(n) = 3n+17$  si  $n < 100$ ;  $317$  si  $n \geq 100 \rightarrow$  tiene orden lineal: VERDADERO**

Para  $n < 100$ ,  $f(n) \leq 3 \cdot 99 + 17 = 314$  (constante). Para  $n \geq 100$ ,  $f(n) = 317$  (constante). Entonces  $f(n) \leq 317$  para todo  $n \geq 1$ , lo que significa que  $f(n) = O(1)$ . Como  $O(1) \subseteq O(n)$ , también es  $O(n)$  (orden lineal). El enunciado es verdadero, aunque la descripción más precisa sería  $O(1)$ .

**e)  $p(n) = 3n^5 + 8n^4 + 2n + 1$  es  $O(n^5) \rightarrow$  VERDADERO**

Para  $n \geq 1$ :  $3n^5 + 8n^4 + 2n + 1 \leq 3n^5 + 8n^5 + 2n^5 + n^5 = 14n^5$ .

Tomando  $C = 14$  y  $n_0 = 1$ :  $p(n) \leq 14 \cdot n^5$  para todo  $n \geq 1$ . Luego  $p(n) = O(n^5)$ .

**f) Si  $p(n)$  es polinomio de grado  $k$ , entonces  $p(n)$  es  $O(n^k) \rightarrow$  VERDADERO**

Sea  $p(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0$ . Para  $n \geq 1$ , cada término  $a_i n^i \leq |a_i| \cdot n^k$ . Sumando todos los términos:  $p(n) \leq (|a_k| + |a_{k-1}| + \dots + |a_0|) \cdot n^k = C \cdot n^k$ . Tomando  $C =$  suma de los valores absolutos de los coeficientes y  $n_0 = 1$ , se cumple  $p(n) = O(n^k)$ .

## Ejercicio 5 – Permutaciones random

**a) ¿Terminan o pueden quedar en loop infinito?**

**randomUno:** puede quedar en loop infinito en teoría, ya que el while busca un número no repetido usando `ran_int` que puede devolver siempre valores ya usados. En la práctica termina, pero no está garantizado formalmente.

**randomDos:** también puede quedar en loop infinito por la misma razón: cuando casi todas las posiciones están usadas, `ran_int` puede seguir generando valores ya marcados indefinidamente.

**randomTres:** SIEMPRE termina. No tiene ningún loop potencialmente infinito: ambos for tienen límite fijo  $n$ , y `ran_int(0, i-1)` siempre genera un índice válido para el swap.

**b) Cantidad de operaciones**

**randomUno:** Para cada posición  $i$ , genera números random hasta encontrar uno no repetido y luego lo compara con los  $i$  anteriores. En el peor caso realiza  $O(N^2)$  operaciones (similar al problema del cumpleaños). No tiene orden determinístico garantizado.

**randomDos:** Usa un array de booleanos para verificar si un número fue usado ( $O(1)$  por verificación), pero el while puede repetirse muchas veces al final cuando quedan pocos números. Promedio:  $O(N \log N)$ , pero sin garantía de terminar.

**randomTres:** Primero llena el array en  $O(N)$ , luego hace exactamente  $N-1$  swaps en  $O(N)$ . Total:  $O(N)$ . Es el más eficiente y el único que garantiza terminación. Implementa el algoritmo de Fisher-Yates.

## Ejercicio 6 – Cálculos con órdenes

### a) Algoritmo $O(\log_2 n)$ , 1 hora $\rightarrow n=1024$ . ¿Con 4 horas?

Si en 1 hora se procesa  $n=1024 = 2^{10}$ , el tiempo es proporcional a  $\log_2(n) = 10$  unidades de tiempo.

Con 4 horas disponemos de  $4 \times 10 = 40$  unidades de tiempo.

Entonces:  $\log_2(n') = 40 \rightarrow n' = 2^{40} = 1.099.511.627.776 \approx 1,1 \times 10^{12}$

Con 4 horas podemos procesar entradas de tamaño aproximadamente 1 billón.

### b) $T(n) = n^2$ , 10.000 op/seg, $n=2.000$

$T(2000) = 2000^2 = 4.000.000$  operaciones.

Tiempo =  $4.000.000 / 10.000 = 400$  segundos  $\approx 6$  minutos 40 segundos.

## Ejercicio 7 – Análisis de algoritmos iterativos

### Algoritmo 1

```
for i = 0 to n+100:           // n+100 iteraciones
    for j = 0 to i*n:         // hasta n^2 iteraciones
        sum += j              // O(1)
    for k = 0 to 3n:          // 3n iteraciones
        c[k] += sum           // O(1)
```

El loop exterior va de 0 a  $n+100 \approx n$  veces. El primer loop interior hace  $i \cdot n$  iteraciones. Sumando sobre  $i$  de 0 a  $n$ :  $\sum(i \cdot n) = n \cdot n(n+1)/2 \approx n^3/2$ . El segundo loop interior hace  $3n$  iteraciones, ejecutado  $n$  veces:  $3n^2$ . El término dominante es  $n^3/2$ .

$T(n) \approx n^3/2 + 3n^2 \rightarrow O(n^3)$

### Algoritmo 2

```
for i = 0 to n^2 (paso 2):    // n^2/2 iteraciones
    for j = n to 1 (paso -n/4): // 4 iteraciones fijas
        x++
```

El loop exterior da  $n^2/2$  vueltas (paso de 2). El loop interior va de  $n$  a 1 restando  $n/4$  en cada paso: hace exactamente 4 iteraciones ( $n, 3n/4, n/2, n/4 \rightarrow$  llega a 0). Son 4 iteraciones fijas independientes de  $n$ .

$T(n) = (n^2/2) \cdot 4 = 2n^2 \rightarrow O(n^2)$

### Algoritmo 3

```
for i = 1 to n (paso 2):      // n/2 iteraciones (i = 1,3,5,...)
    for j = 1 to i:          // i iteraciones
        r++
```

El loop exterior toma valores impares: 1, 3, 5, ..., n-1 (aproximadamente n/2 valores). Para cada i, el loop interior hace i iteraciones. La suma de los primeros n/2 números impares es aproximadamente  $(n/2)^2 = n^2/4$ .

**$T(n) \approx n^2/4 \rightarrow O(n^2)$**

### Ejercicio 8 – plter vs pRec (potenciación)

#### a) ¿Qué hacen?

Ambos métodos calculan  $x^n$  (x elevado a la n).

#### b) Función de tiempo y orden

**plter(x, n):** iterativo. Si n=0 retorna 1; si n=1 retorna x; si n>1 hace un loop de i=2 hasta n multiplicando por x en cada paso: n-1 multiplicaciones.

**$T(n) = n - 1 \rightarrow O(n)$**

**pRec(x, n):** recursivo por exponenciación rápida. Si n es par: pRec( $x^2$ , n/2). Si n es impar: pRec( $x^2$ , n/2) · x. En cada llamada n se divide a la mitad.

Recurrencia:  $T(n) = T(n/2) + O(1)$ . Resolviendo:  $T(n) = O(\log_2 n)$ .

**$T(n) = \log_2 n \rightarrow O(\log n)$**

#### c) ¿Cuál es más eficiente?

pRec es más eficiente:  $O(\log n)$  vs  $O(n)$ . Para n=1.000.000, plter hace ~1.000.000 operaciones, pRec hace solo ~20.

### Ejercicio 9 – rec1, rec2, rec3

#### a) Funciones de tiempo

**rec1(n):** llama dos veces a rec1(n-1). Árbol binario completo de profundidad n.

$T(n) = 2 \cdot T(n-1) + 1$ . Resolviendo:  $T(n) = 2^n - 1$ .

**Orden:  $O(2^n)$  (exponencial)**

**rec2(n):** llama una sola vez a rec2(n-1) y multiplica por 2.

$T(n) = T(n-1) + 1$ . Resolviendo:  $T(n) = n$ .

**Orden:  $O(n)$  (lineal)**

**rec3(n):** llama dos veces a  $\text{rec3}(n-2)$ . Árbol binario de profundidad  $n/2$ .

$T(n) = 2 \cdot T(n-2) + 1$ . Resolviendo:  $T(n) = 2^{(n/2)}$ .

**Orden:  $O(2^{(n/2)}) = O(\sqrt{2^n})$  (exponencial)**

## b) Comparación rec1 vs rec2

Aunque ambos calculan lo mismo ( $2^{(n-1)}$ ),  $\text{rec1}$  es  $O(2^n)$  porque hace 2 llamadas recursivas por nivel (árbol con  $2^n$  nodos), mientras que  $\text{rec2}$  es  $O(n)$  porque hace una sola llamada por nivel (cadena lineal).  $\text{rec2}$  es drásticamente más eficiente.

## c) Alternativa más eficiente que rec3

$\text{rec3}$  llama dos veces a  $\text{rec3}(n-2)$ , lo que genera un árbol exponencial. La mejora consiste en llamar una sola vez y guardar el resultado:

```
static public int rec3Eficiente(int n) {  
    if (n == 0) return 0;  
    if (n == 1) return 1;  
    int mitad = rec3Eficiente(n-2);  
    return mitad * mitad;  
}
```

Ahora  $T(n) = T(n-2) + O(1)$ , que resuelve a  $T(n) = O(n/2) = O(n)$ .

## Ejercicio 10 – Tiempo teórico de búsquedas (compara con Ej 2)

**buscarLineal:** recorre el array elemento por elemento. En el peor caso (elemento al final o no existe), revisa todos los  $N$  elementos.

**$T(n) = n \rightarrow O(n)$**

**buscarDicotomica:** en cada iteración descarta la mitad del rango. Desde  $n$  elementos llega a 1 en  $\log_2(n)$  pasos.

**$T(n) = \log_2(n) \rightarrow O(\log n)$**

Comparación con el ejercicio 2: los tiempos empíricos confirman esto. La búsqueda lineal hizo exactamente  $N$  iteraciones en cada caso (100.000, 200.000, etc.), y la dicotómica hizo  $\log_2(N)$  iteraciones (16, 17, 18, 19). Coincide perfectamente con la teoría.

## Ejercicio 11 – Tiempo teórico de procesarMovimientos (compara con Ej 1)

**procesarMovimientos:** doble loop: Q consultas × hasta N cuentas por rango.

$$T(n) = N \times Q = N^2 \text{ (con } N = Q) \rightarrow O(N^2)$$

**procesarMovimientosOptimizado:** un loop de Q consultas (2 ops cada una) + un loop de N cuentas.

$$T(n) = 2Q + N = 3N \text{ (con } N = Q) \rightarrow O(N)$$

Comparación con ejercicio 1: los tiempos empíricos lo confirman. De N=50.000 a N=100.000 (doble), procesarMovimientos tardó de 15.4 a 71 seg (~4.6x, consistente con x4 de  $O(N^2)$ ). El optimizado pasó de 0.016 a 0.015 seg (prácticamente igual, consistente con  $O(N)$  a esta escala).

## Ejercicio 12 – Funciones recursivas

**recur(n) – divide n a la mitad, llama 2 veces**

```
return recur(n/2) * recur(n/2)
```

$T(n) = 2 \cdot T(n/2) + 1$ . Por el teorema maestro ( $a=2$ ,  $b=2$ ,  $f(n)=1$ ):  $\log_b(a) = \log_2(2) = 1 > 0$ .

$T(n) = O(n)$  (caso 1 del teorema maestro)

**recursivo(n) – resta 5 en cada llamada**

```
return recursivo(n-5)
```

$T(n) = T(n-5) + 1$ . Se hacen  $n/5$  llamadas hasta llegar al caso base ( $n \leq 5$ ).

$T(n) = n/5 \rightarrow O(n)$

**recursivo(n) – divide por 8 en cada llamada**

```
return recursivo(n/8)
```

$T(n) = T(n/8) + 1$ . Se hacen  $\log_8(n)$  llamadas hasta llegar al caso base ( $n \leq 7$ ).

$T(n) = \log_8(n) \rightarrow O(\log n)$

## Ejercicio 13 – Análisis y tiempo de ejecución para n=4096

```
for i = 1; i <= n; i = i*2: // log2(n) iteraciones
    for j = 0; j < n; j += n/2: // 2 iteraciones fijas (j=0 y j=n/2)
        a[j]++
```

El loop exterior va de 1 hasta n duplicando i: 1, 2, 4, 8, ..., n  $\rightarrow \log_2(n)$  iteraciones.

El loop interior va de 0 hasta n con paso n/2: solo toma los valores 0 y n/2  $\rightarrow 2$  iteraciones siempre.

$$T(n) = \log_2(n) \times 2 = 2 \cdot \log_2(n) \rightarrow O(\log n)$$

Para  $n = 4096 = 2^{12}$ :

$$T(4096) = 2 \times \log_2(4096) = 2 \times 12 = 24 \text{ operaciones.}$$

Con 1.000 operaciones/segundo: tiempo =  $24 / 1.000 = 0.024$  segundos.

## Ejercicio 14 – Función recursiva con while

```
while (i <= n) { i *= 2 }    // log2(n) iteraciones
return n + recursivo(n/2) * n/2 // 1 llamada recursiva
```

El while dentro de la función hace  $\log_2(n)$  iteraciones (i duplica hasta superar n).

La función se llama recursivamente con n/2, y así hasta  $n \leq 1$ .

Recurrencia:  $T(n) = T(n/2) + \log_2(n)$ .

Expandiendo:  $T(n) = \log(n) + \log(n/2) + \log(n/4) + \dots = \log(n) + (\log(n)-1) + (\log(n)-2) + \dots \approx (\log n)^2/2$ .

$$T(n) = O(\log^2 n)$$

## Ejercicio 15 – Doble loop con rangos variables

```
for i = n2; i <= n3; i += n2: // n iteraciones (i toma n2, 2n2, ..., n3)
    for j = 1; j <= i; j++:      // i iteraciones
        resta -= j
```

El loop exterior: i comienza en  $n^2$ , termina en  $n^3$ , con paso  $n^2$ . Cantidad de iteraciones:  $(n^3 - n^2)/n^2 + 1 = n - 1 + 1 = n$  iteraciones. Los valores de i son:  $n^2, 2n^2, 3n^2, \dots, n \cdot n^2 = n^3$ .

El loop interior para cada valor de i hace i iteraciones.

Sumando el trabajo total:  $\sum (k \cdot n^2)$  para  $k=1$  hasta  $n = n^2 \cdot n(n+1)/2 \approx n^4/2$ .

$$T(n) = n^4/2 \rightarrow O(n^4)$$

## Ejercicio 16 – Loop con while compartido

```
for i = 0; i < n; i++:
    while j < n && arr[i] < arr[j]:
```



```
j++
```

La clave es que  $j$  es una variable compartida entre todas las iteraciones del for, no se reinicia. Comienza en 0 y solo puede llegar hasta  $n$ . El while hace que  $j$  avance, pero  $j$  nunca retrocede. En total,  $j$  puede incrementarse como mucho  $n$  veces a lo largo de todo el for.

Entonces: el for hace  $n$  iteraciones + el while hace en total a lo sumo  $n$  incrementos de  $j$ .

$$T(n) = n + n = 2n \rightarrow O(n)$$

## Ejercicio 17 – Doble loop con $j = i \% n$

```
for i = 0; i < n2; i++:           // n2 iteraciones
    for j = i%n; j >= 0; j--:      // (i%n + 1) iteraciones
        println(j)
```

El loop exterior hace  $n^2$  iteraciones. Para cada  $i$ , el loop interior hace  $(i \% n) + 1$  iteraciones.  $i \% n$  toma los valores 0, 1, 2, ...,  $n-1$ , 0, 1, 2, ...,  $n-1$ , ... (ciclo de período  $n$ , repetido  $n$  veces).

Suma del trabajo del loop interior en un ciclo:  $\sum(k+1)$  para  $k=0$  hasta  $n-1 = 1+2+\dots+n = n(n+1)/2 \approx n^2/2$ .

Esto se repite  $n$  veces ( $n$  ciclos completos):  $n \times n^2/2 = n^3/2$ .

$$T(n) = n^3/2 \rightarrow O(n^3)$$

## Ejercicio 18 – Triple loop anidado

```
while i <= n:                       // n/2 iteraciones (i de 1 a n, paso 2)
    while j > 1:                     // log2(n) iteraciones (j = n, n/2, n/4, ...)
        for k = i to i+10:          // 10 iteraciones fijas
            println(...)
        j = j/2
    i += 2
```

Atención:  $j$  se inicializa en  $n$  ANTES del while exterior y NO se reinicia en cada iteración del while exterior. Esto es crucial.

El while exterior ( $i$ ) itera  $n/2$  veces. Pero  $j$  solo se divide hasta llegar a 1 una sola vez ( $\log_2(n)$  divisiones en total a lo largo de TODO el while exterior, no por cada  $i$ ).

El for interior siempre hace 10 operaciones.

Trabajo total del while interior + for:  $\log_2(n) \times 10 = 10 \cdot \log_2(n)$  (en total, no por iteración de  $i$ ).

Más el trabajo del while exterior sin contar el interior:  $n/2$ .

$$T(n) = n/2 + 10 \cdot \log_2(n) \rightarrow O(n)$$

— Fin de la Práctica 4 —